



Universidad Nacional Autónoma de México

Facultad de Ciencias

Ciencias de la Computación

Práctica 6: Verificación de tipos en MiniC

Compiladores

Semestre 2027-1

Equipo: 02

Integrantes: Flores Galeana Daniela - 321162342
Rivera Machuca Gabriel Eduardo - 321057608

Profesora: L. en C.C. Yessica Janeth Pablo Martínez
Ayudante: Pedro Yamil Salazar González

Fecha de entrega: 01 de noviembre del 2026

Índice

1. Objetivo **2**

2. Evolución respecto de la Práctica 5 **2**
2.1. Módulos modificados 2
2.2. Flujo del compilador 2

3. Sistema de tipos y representación interna **2**

4. Recorrido del AST y resolución de nombres **3**

5. Validación de tipos **3**
5.1. Inicializaciones y asignaciones 3
5.2. Operadores 3
5.3. Condiciones e impresión 4

6. Diagnósticos y recuperación **4**

7. Propiedad y manejo de memoria **4**

8. Resultados y pruebas **4**
8.1. Pruebas de memoria 5

9. Problemas encontrados **5**

10. División del trabajo **6**

11. Uso de herramientas de Inteligencia Artificial **6**

12. Conclusiones **6**

13. Referencias **7**

1. Objetivo

El objetivo principal de esta sexta práctica fue extender el analizador semántico de MiniC para verificar los tipos de las expresiones y su compatibilidad en declaraciones, asignaciones y estructuras de control. Para esto se reutilizó el Árbol de Sintaxis Abstracta (AST), la tabla de símbolos y el manejo de ámbitos de la práctica anterior. La verificación se realiza después de los análisis léxico y sintáctico, sin evaluar expresiones ni generar código.

2. Evolución respecto de la Práctica 5

2.1. Módulos modificados

Se extendió el recorrido del módulo `src/semantic/semantic.c` para determinar el tipo de las expresiones y revisar su uso en las sentencias. La enumeración de tipos se conservó en `include/semantic/symbol.h` y se agregó la función `type_to_string`, que permite escribir los nombres de los tipos de manera consistente en los diagnósticos.

También se corrigió el mapeo previo de las declaraciones booleanas, que devolvía `TYPE_INT` en lugar de `TYPE_BOOL`. El lexer, el parser, el AST y la tabla de símbolos se conservaron, con ajustes en el manejo de memoria y errores internos.

El Makefile registra las dependencias de los encabezados y permite ejecutar todas las baterías funcionales mediante `make test`. Las pruebas oficiales del material de la práctica 6 se conservaron por separado en `tests/public_p06/`, sin sustituir las pruebas anteriores.

2.2. Flujo del compilador

El ejecutable `minic` sigue recibiendo un archivo mediante `./minic archivo.mc`. Si existen errores léxicos o sintácticos no se ejecuta el análisis semántico. Cuando el programa es válido se imprime únicamente `Programa semanticamente correcto.` en `stdout` y se devuelve cero. Los diagnósticos se escriben en `stderr`; ante errores, `stdout` permanece vacío y el código de salida es distinto de cero.

3. Sistema de tipos y representación interna

MiniC maneja dos tipos declarables, `int` y `bool`, sin conversiones implícitas entre ellos. La representación interna utiliza los siguientes valores:

- `TYPE_INT`: Corresponde a expresiones enteras.
- `TYPE_BOOL`: Corresponde a expresiones booleanas.
- `TYPE_ERROR`: Indica que el análisis de una expresión encontró un error semántico.
- `TYPE_UNKNOWN`: Representa un tipo no determinado o no aplicable.

La función `visit_expression` devuelve el tipo de cada expresión durante el recorrido del árbol. Los tipos se propagan desde las hojas hacia los operadores que las contienen. Se eligió devolver el resultado en lugar de agregar un campo al AST, ya que el recorrido existente permitía hacer la verificación sin guardar nuevas referencias a símbolos. Por esta decisión, el tipo inferido no queda almacenado en el árbol.

4. Recorrido del AST y resolución de nombres

El analizador procesa las sentencias en el orden en que aparecen. Los literales enteros producen `TYPE_INT` y los literales `true` y `false` producen `TYPE_BOOL`. En el caso de un identificador, `symbol_table_lookup` busca desde el ámbito actual hacia sus padres y devuelve el tipo declarado del símbolo visible más cercano.

La búsqueda local se sigue utilizando para detectar redeclaraciones dentro del mismo ámbito. Los bloques anidados permiten el ocultamiento de nombres (*shadowing*); por ejemplo, una variable booleana local puede ocultar a una variable entera exterior sin cambiar el tipo de esta última.

5. Validación de tipos

5.1. Inicializaciones y asignaciones

Al analizar una declaración se registra el símbolo antes de recorrer su inicializador. Si hay un inicializador, su tipo debe coincidir exactamente con el tipo escrito en la declaración. Cuando existe una redeclaración se conserva el símbolo anterior, se informa el error y se revisa el inicializador usando el tipo escrito en la nueva declaración.

En una asignación se busca el símbolo visible del destino y se compara su tipo con el de la expresión derecha. Una asignación incompatible no cambia el tipo declarado de la variable. Si el destino no existe, se reporta el identificador no declarado y se analiza la expresión derecha para encontrar errores independientes, sin comparar con un tipo de destino inexistente.

5.2. Operadores

Las reglas implementadas para las expresiones son:

- **Aritméticos:** `+`, `-`, `*` y `/` requieren dos operandos `int` y producen `int`. El menos unario requiere y produce `int`.
- **Relacionales:** `<`, `<=`, `>` y `>=` requieren enteros y producen `bool`.
- **Igualdad:** `==` y `!=` requieren operandos del mismo tipo y producen `bool`.
- **Lógicos:** `&&` y `||` requieren dos operandos `bool` y producen `bool`.

La precedencia y asociatividad se conservan de la estructura del AST construida por el parser. La verificación utiliza los tipos de los operandos y no calcula sus valores.

5.3. Condiciones e impresión

Las condiciones de `if` y `while` deben producir `bool`. Aunque la condición sea inválida, se revisan el cuerpo y las ramas correspondientes. La sentencia `print` acepta expresiones válidas de tipo `int` o `bool`.

6. Diagnósticos y recuperación

Los errores semánticos se reportan en `stderr` con categoría, línea y columna. En las incompatibilidades se identifica la construcción o el operador y se indican los tipos esperados y encontrados. Se utiliza la posición del operador, la del identificador de declaración o asignación, o la raíz de la condición, según el caso; las columnas comienzan en cero.

Cuando un identificador no está declarado, su expresión devuelve `TYPE_ERROR`. Si un operador recibe este resultado, también devuelve `TYPE_ERROR` sin agregar una incompatibilidad derivada. La misma regla se aplica al comparar inicializadores, asignaciones y condiciones. Los dos operandos se recorren para detectar errores independientes, y el análisis continúa con las sentencias y ramas restantes cuando el error es recuperable.

7. Propiedad y manejo de memoria

El AST es dueño de sus nodos y cadenas. La tabla de símbolos posee los ámbitos creados; cada ámbito posee sus símbolos, y cada símbolo mantiene una copia propia de su nombre. La referencia desde un símbolo hacia su declaración en el AST es prestada. El analizador de tipos no agrega referencias desde el AST hacia símbolos; al terminar se destruye primero la tabla y después el árbol.

Los constructores del AST adquieren sus hijos o listas únicamente si terminan correctamente. Si una reserva falla, el parser conserva la responsabilidad de liberar los recursos que todavía le pertenecen, reporta un error interno y detiene el análisis. También se corrigió la liberación de copias temporales de nombres en el parser y del AST cuando falla el cierre del archivo.

8. Resultados y pruebas

La compilación se realizó con GCC mediante `make`, usando `-std=c11 -Wall -Wextra -Wpedantic`, sin advertencias. Se conservaron las pruebas públicas y las del equipo, y se agregaron diez casos en `tests/tipos/`. Los casos incluyen:

- Literales e identificadores de ambos tipos, inicializaciones y asignaciones válidas e incompatibles.
- Operadores aritméticos, unario, relacionales, de igualdad y lógicos, con expresiones anidadas, precedencia y asociatividad.

- Condiciones válidas e inválidas, *shadowing*, variables no declaradas, errores independientes y prevención de cascadas.
- Errores léxicos y sintácticos, recuperación entre sentencias inválidas y un programa completamente válido.
- Expresiones que no deben evaluarse durante el análisis, como una división entre cero, un entero extenso y un ciclo que no termina al ejecutarse.

Las diez pruebas de tipos comparan `stdout`, `stderr` y el código de salida con los archivos esperados. Se obtuvieron resultados satisfactorios en esos diez casos, en los diez casos del equipo mediante `make test_eq`, y en las 24 comprobaciones automáticas de las pruebas públicas heredadas.

También se revisaron la guía de integración y el checklist de `Material de la práctica 6`. Se incorporaron sus 24 casos oficiales, que se ejecutan con `make test_p06`. Todos cumplieron el contrato de salida. Además de aceptar o rechazar cada programa, se contrastaron las causas, los tipos, las posiciones y la ausencia de errores en cascada con el archivo `esperados/CASOS.md` del material.

En total, `make test` ejecuta 68 casos de las cuatro baterías. Los dos ejemplos adicionales del material también produjeron los resultados esperados: el programa válido mostró únicamente el mensaje de éxito y el inválido reportó seis errores independientes. Estos resultados no garantizan por sí solos una calificación ni cubren todos los programas posibles.

El caso público `p20_types_deferred` se actualizó para esperar un rechazo, ya que en la práctica anterior las incompatibilidades de tipos estaban fuera de alcance. También se corrigió la salida esperada de `p3` del equipo y el salto de línea faltante en la salida de `p9`.

8.1. Pruebas de memoria

Una ejecución inicial con Valgrind sobre el programa válido reportó 864 bytes perdidos en 54 bloques. Se encontró que el lexer conservaba un búfer temporal después de que `token_init` copiaba su contenido. Tras liberar ese búfer, Valgrind 3.27.1 informó cero bloques pendientes y cero errores en el caso válido. También se ejecutaron con Valgrind los 24 casos oficiales y los dos ejemplos del material, sin informes de fugas ni errores de memoria.

Con `make test_memoria` se simularon 307 fallos individuales de reserva alcanzados por el programa válido. Se comprobaron los diagnósticos, los códigos de salida y la memoria con Valgrind, sin informes de errores de memoria. Estas pruebas no abarcan todas las entradas ni todas las rutas posibles.

9. Problemas encontrados

Durante la revisión de memoria también se encontró la fuga de los búferes temporales del lexer. Además, algunas rutas del parser no liberaban los subárboles cuando fallaba

una reserva. Se ajustó la propiedad de los recursos y se añadieron pruebas de fallos simulados para las reservas alcanzadas por el programa válido.

En la revisión final se detectó que la recuperación sintáctica podía detenerse antes de revisar otra sentencia inválida. Se corrigió la coordinación de la recuperación entre las listas de sentencias y se agregó una prueba con varios errores independientes. También se corrigió un fallo de reserva del lexer que se informaba como error de lectura y se evitó truncar los diagnósticos de identificadores largos.

10. División del trabajo

Como propuesta para la revisión final de la práctica, se distribuyen las actividades de la siguiente manera:

- **Gabriel Eduardo Rivera Machuca:** Revisión de la representación de tipos y su relación con la tabla de símbolos y los ámbitos. Revisión de las reglas de operadores, *shadowing* y pruebas de expresiones.
- **Daniela Flores Galeana:** Revisión del recorrido semántico, los diagnósticos y la propagación de `TYPE_ERROR`. Revisión de inicializaciones, asignaciones, condiciones y pruebas de manejo de memoria.

11. Uso de herramientas de Inteligencia Artificial

Para esta práctica se utilizó Gemini (Google) como herramienta de apoyo técnico y consulta.

- **Apoyo en el reporte:** .ªyudame a organizar este reporte anexado y a mejorar la redacción"
- **Consulta sobre memoria:** "por qué me sale: `definitely lost: 864 bytes in 54 blocks`.?"
- **Apoyo recibido:** La herramienta explicó que el lexer no liberaba los búferes temporales después de copiar los lexemas al token. También ayudó a corregir la fuga, revisar los fallos de reserva y ejecutar las pruebas. Además, apoyó en el análisis y modificación del código de la práctica y en la preparación de pruebas.
- **Revisión y comprobación:** Se compilaron y ejecutaron las pruebas descritas en este reporte. La revisión y adaptación del código y del reporte por ambos integrantes queda pendiente antes de la entrega.

12. Conclusiones

La verificación de tipos amplió el análisis semántico de MiniC para detectar operaciones que tienen una estructura sintáctica válida, pero usan tipos incompatibles. El

recorrido del AST y la búsqueda de símbolos visibles permiten relacionar cada expresión con las declaraciones y ámbitos correspondientes.

El uso de `TYPE_ERROR` ayudó a evitar diagnósticos en cascada y permitió seguir revisando errores independientes. Las pruebas realizadas muestran el comportamiento de las reglas y del manejo de memoria en los casos ejecutados, aunque no sustituyen la revisión del equipo ni cubren todos los programas posibles.

13. Referencias

- Facultad de Ciencias, UNAM. (2026). *Material de apoyo de la Práctica 6*. Guía de integración, checklist, pruebas públicas y ejemplos del archivo `Material_Practica6.zip`.
- Free Software Foundation. (s.f.). *Using the GNU Compiler Collection: Options to Request or Suppress Warnings*. `https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html`
- Valgrind Developers. (s.f.). *Valgrind User Manual: Memcheck, a memory error detector*. `https://valgrind.org/docs/manual/mc-manual.html`